

Formal Methods for System Verification

Dr. Chandan Karfa

Department of Computer Science and Engineering



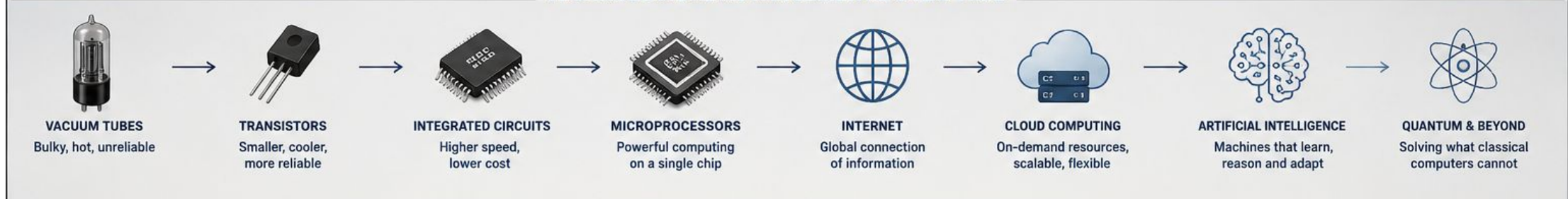
ভাৰতীয় প্রযুক্তিবিদ্যা প্রতিষ্ঠান গুৱাহাটী
भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

THE EVOLUTION OF ELECTRONIC COMPUTING

From Early Machines to Intelligent Systems (1940s–2026)

1940s The Beginning	1950s First Generation	1960s Second Generation	1970s Third Generation	1980s Fourth Generation	1990s Personal Computing & Internet Era	2000s Mobile & Connected World	2010s Cloud, Big Data & Smart Devices	2020s Intelligent & Autonomous Era	2026 The Next Frontier
									
ENIAC (1945) • Vacuum tubes • Huge size • Thousands of connections • Programmed in machine code	UNIVAC I (1951) • Vacuum tubes • Magnetic cores • Assembly language • Used for business, census, and defense	IBM 1401 (1959) • Transistors • Smaller, faster, more reliable • High-level languages (FORTRAN, COBOL) • Batch processing	PDP-8 (1965) • Integrated Circuits (ICs) • Minicomputers • Timesharing • Operating Systems (early UNIX)	IBM PC (1981) • Microprocessors (LSI/VLSI) • Personal Computers • GUIs begin (Windows, Mac) • Spreadsheets, Word Processing, Databases	Gateway 486 (1993) • Affordable PCs • Internet growth • Email, Browsers • Client–Server Computing • Multimedia	• Laptops, Smartphones • Wi-Fi, Broadband • Cloud Computing begins • Social Media • Virtualization	• Cloud Computing mainstream • Big Data Analytics • AI/ML advances • IoT (Internet of Things) • Touch, Voice, Apps	• AI Everywhere • Edge Computing • 5G/6G Connectivity • Autonomous Systems • Quantum Computing Progress	• Human–AI Collaboration • Quantum Computing Scaling Up • Neuromorphic & Brain–Computer Interfaces • Ambient Intelligence • Sustainable & Green Computing • Computing becomes Invisible, Ubiquitous and Intelligent

TECHNOLOGICAL ADVANCEMENTS OVER TIME

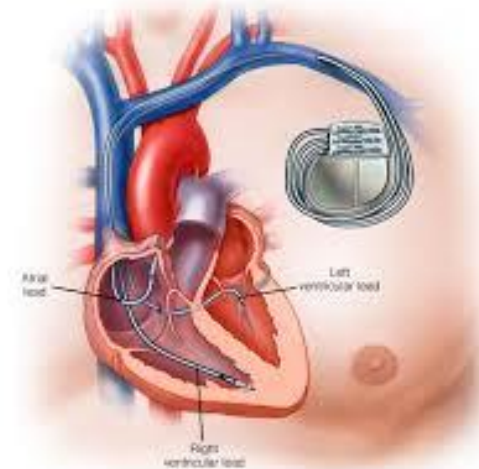
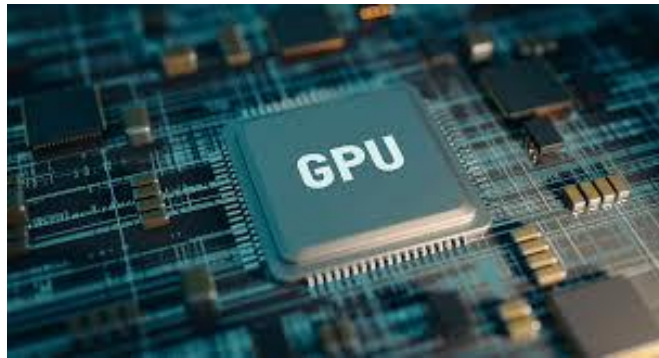


IMPACT ON SOCIETY



“From Calculators to Collaborators, Computing Powers Progress.” **The Future is Intelligent.**

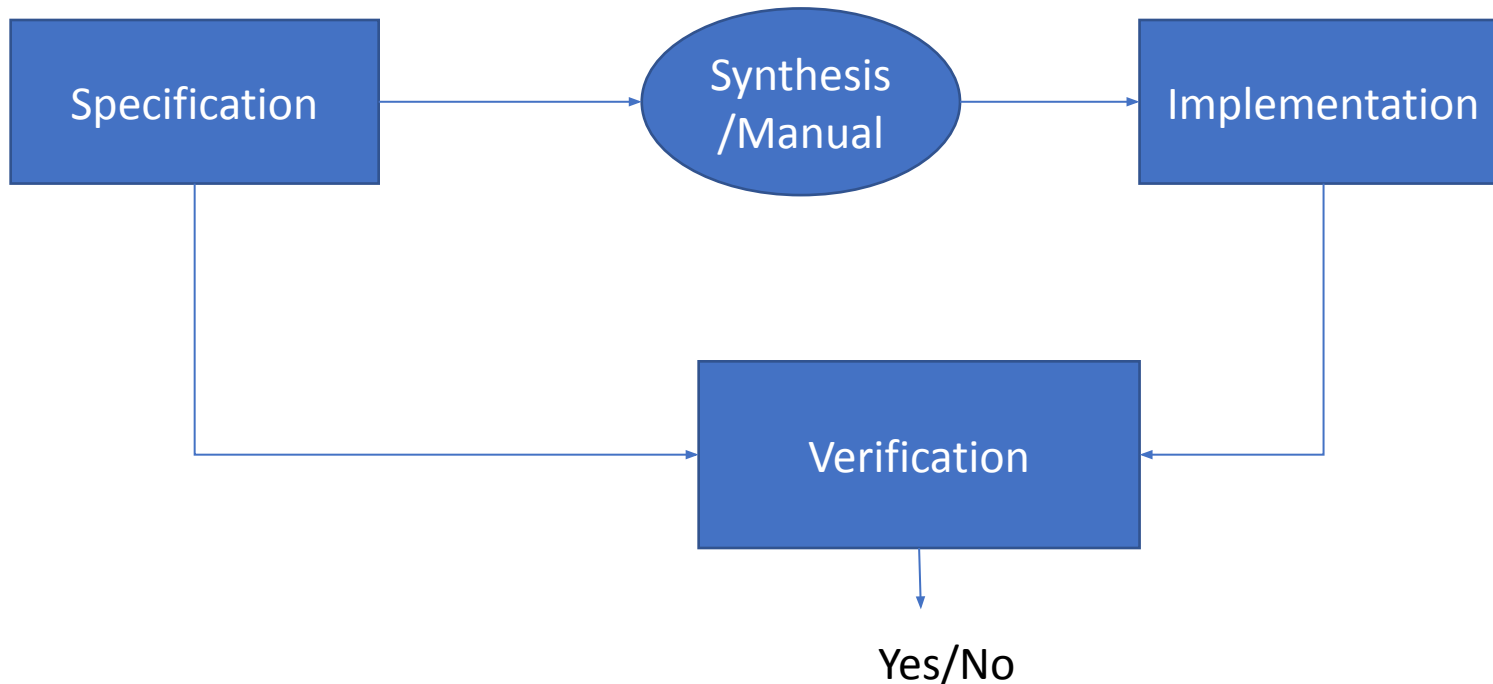
What if the System fails in a very Specific case?



As computing evolves, ensuring system reliability becomes as important as innovation itself.

What do we Verify?

- The implementation correctly implements the specification.
- Is my implementation correct?



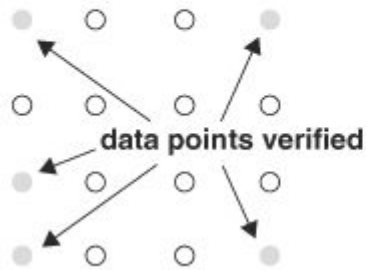
How do we verify?

- Simulation based Verification
 - Based on set of inputs
- Formal verification
 - Mathematically without any inputs

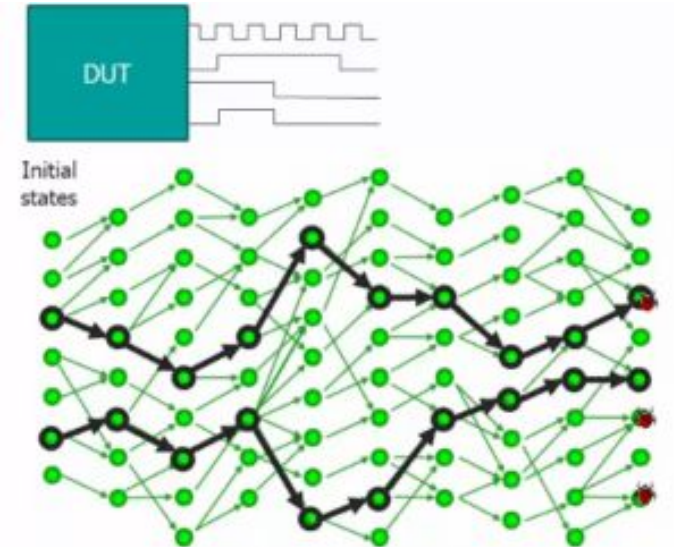
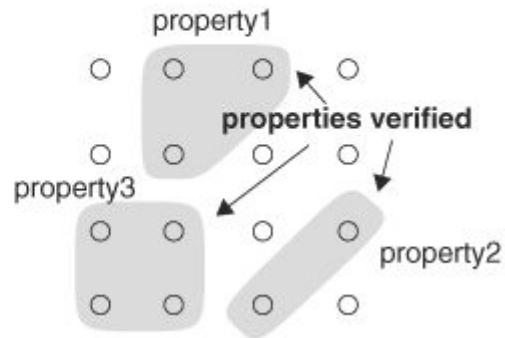
Why do we need Formal Methods for Verification?

- Simulation based verification vs Formal Verification

simulation-based verification



formal verification



How do we formally Verify?

Applies mathematics for modelling and Analyzing Systems

- Theorem Proving
 - SAT Solver
 - SMT Solver
- Model Checking
 - CTL
 - LTL
- Equivalence Checking
 - Translation validation

Course Planning

Part 1: Theorem Proving

- Spectrum of Logic
- Decision Problems of Propositional Logic
- Introduction to SAT Solver
 - DPLL based SAT Solver
- Introduction to SMT Solver
- ROBDD

• Part 3: Equivalence Checking

- C to RTL Equivalence Checking
- Verification of RTL Optimizations

• Part 4: LLM in Formal Verification

• Part 2: Formal Property Checking

- Introduction to Model Checking
- LTL Model Checking
- CTL Model Checking
- SAT Based Model Checking
- BDD based model Checking
- Bounded Model Checking

Who can Attend this Course?

- BTech/MTech in CSE, ECE, EEE interested in Verification
- Research Scholars working in Formal Verification
- Verification Engineers
- EDA Tool Developers

